laravel-workflow 2.4

A step-by-step guide to adding this Claude Code workflow to your Laravel app, and what it does once it's in.

---

1. What is laravel-workflow?
2. What you need first
3. Installing it: plugin or project files
4. Laravel Boost (recommended)
5. Your first task: plan, approve, build
6. How a task is checked
7. Running many tasks safely
8. Git: one command to commit, pull, push
9. All commands and helpers
10. The specialist agents
11. Automatic checks (hooks)
12. Post and Page content: Gutenberg
13. Redis, uploads, S3 and CDN
14. Settings you can change
15. A typical day
16. Problems and fixes
17. What's new in 2.4 and 2.3
18. Measuring the workflow

1. What is laravel-workflow?

laravel-workflow is a **kit for Claude Code**, not a Composer package. Nothing is added to your app's code. It adds instructions and checks to Claude Code, as a plugin or as files in your project's `.claude/` folder, and they change how Claude Code works on your Laravel project.

- **It plans before it builds.** Opus turns your request into small draft task files with clear "done" checks. You read and approve them.
- **Sonnet builds each task**, directly for small tasks and with specialist agents for large ones.
- **A second agent checks the work.** It sees only the task and the changes, never the builder's reasoning. A task can't be closed without its verdict.
- **Automatic checks** catch the mistakes that only show up in production: `env()` outside config, mass assignment, raw SQL, a missing `@csrf`, N+1 queries, and more.
- **It knows Laravel 13** (rules checked against laravel.com), and it uses Laravel Boost when you have it.
- **Post and Page modules** always use the Gutenberg block editor, never CKEditor.
- **Pure code.** Nothing written for a task carries a comment; intent goes into names, types and small methods. A check blocks any edit that adds one.
- **The decisions stay yours.** Claude can't approve its own drafts, accept work nobody could check, or push.
- **It's safe to undo.** You can get one commit per task and roll back any single task.

> **In one sentence**
>
> Describe what you want with `/laravel:write-task`, approve the plan, run `/laravel:task`, and every task is built, independently checked and recorded.

2. What you need first

Requirement	Why
A Laravel 11, 12 or 13 app	It needs `artisan` and `laravel/framework` in `composer.json`. The kit detects the version.
Claude Code	The workflow runs inside it (terminal, desktop app or IDE).
bash, PHP	The checks are shell scripts (Linux, macOS, or WSL on Windows).
`jq`	Needed for safe settings merging and batch runs.
git (recommended)	Enables task diffs, the verifier's view of changes, per-task commits and undo.
Pint, Pest/PHPUnit, PHPStan (optional)	Used if present; nothing is installed for you.
Playwright (optional)	Lets the verifier check `[browser]` criteria in a real browser.

3. Installing it: plugin or project files

There are two ways. Pick **one** per project. If both are present, the plugin says so at session start and the project files win.

	A. Plugin new in 2.4	B. Project files
Where it lives	Once, in Claude Code. Nothing is copied into the project.	Copied into `your-app/.claude/`, so it can be committed for the team.
Updating	`claude plugin update laravel@laravel-workflow`	Run `./install.sh` again.
Helper commands	In the plugin's folder. The session report prints the path ("Kit helpers").	`bash .claude/hooks/…`

A Install as a plugin

```
claude plugin marketplace add ~/www/hobbies/laravel-workflow
claude plugin install laravel@laravel-workflow
```

The commands are the same `/laravel:*`. A plugin can't carry permission rules, so extra checks take their place: one approves only the kit's own helpers, your test, lint and build commands, a few read-only artisan commands and Boost's read-only tools; one blocks Claude from reading `.env` files, keys, `auth.json` and `storage/`. Your own deny and ask rules always win.

B Or copy it into your app

```
cd ~/www/hobbies/laravel-workflow
./install.sh /path/to/your-laravel-app          # add --dry-run to preview
```

This copies the kit into `your-laravel-app/.claude/` and merges `settings.json`. The old file is backed up, older kit entries are replaced, and permission rules that earlier versions shipped and later removed are dropped. If PHPStan is installed, it also records the findings that already exist, so Claude is held only to new ones.

1 **Open Claude Code in your app, then run setup once**

```
cd /path/to/your-laravel-app
claude
/laravel:setup
```

At start you'll see a short report: kit version, Laravel and PHP versions, the view layer, drivers, storage, the Kit helpers path and the verify commands. That means it's working. Setup creates `.claude/tasks/`, fills `.claude/project-context.md` with facts about your repo, adds a short pointer to `CLAUDE.md`, recommends Laravel Boost if it's missing, and tells you what won't work here.

> **If setup says static analysis is already failing**
>
> Type `! bash .claude/hooks/verify.sh --baseline` in the prompt (plugin: the Kit helpers path from the report). That records the existing findings. Claude isn't allowed to do this itself, so it can never excuse its own mistakes.

> **Commit**
>
> With project files: `.claude/` (including `.claude/tasks/`) and `CLAUDE.md`, so your team gets the same workflow. With the plugin: `.claude/tasks/`, `.claude/project-context.md` and `CLAUDE.md`. `.claude/.cache/` is git-ignored.

4. Laravel Boost (recommended)

Laravel Boost is Laravel's official MCP server. With it installed, the kit's agents use its read-only tools:

Boost tool	Used for
`search-docs`	Documentation for your exact package versions, checked before relying on memory
`database-schema`, `database-query`	Reading the real schema and stored data (read-only) instead of guessing
`last-error`, `read-log-entries`, `browser-logs`	Seeing the actual exception when something fails
`record-rule`	Team-shared project rules in `.ai/rules`, used by the main session only, never by agents

```
composer require laravel/boost --dev
php artisan boost:install          # choose Claude Code: guidelines, skills, MCP
```

Claude Code asks once to approve the project's `.mcp.json` server. The read-only Boost tools are then pre-approved (by permission rules with project files, by the plugin's check with the plugin).

5. Your first task: plan, approve, build

1 **Describe what you want**

```
/laravel:write-task an admin screen where editors manage their own news and can delete one
```

The planner (Opus) asks questions first if something is unclear, splits big requests into small tasks, sets the order (`depends`) and size, and writes them as **drafts**. It never writes code.

2 **Read the drafts, then approve them in your own terminal**

```
---
id: 003
title: Editors manage their own news
status: draft
depends: 002
size: large
---
## Goal
Editors see and delete only their own news items in the admin area.

## Acceptance
- [ ] [test]    Deleting another user's news returns 403 and deletes nothing
- [ ] [browser] Guests are redirected to "/"
- [ ] [review]  Authorization lives in a NewsPolicy
```

Each "done" check carries a tag saying how it will be proven:

- `[test]` : a Pest/PHPUnit test must prove it.
- `[browser]` : it has to be seen in the running app.
- `[review]` : reading the code is enough.

```
bash .claude/hooks/tasks.sh approve all          # or: approve 003
```

Approving is your step: `approve` refuses to run inside a Claude Code session, and Claude is blocked from calling it or editing a task's `status:` .

3 **Build it**

```
/laravel:task
```

Sonnet takes the next runnable task and works through it: builds it (directly if small, with the full phased build if large), runs the checks, gets the verifier's verdict (section 6), closes it with a log line, and saves what it learned.

draft → you approve → **todo** → **doing** → **done** or **review** or **blocked**

6. How a task is checked

The builder can't mark its own work done. `tasks.sh close` refuses until a verdict is recorded: **new in 2.3**

Verdict	What happens
PASS	Every criterion was proven. The task closes as `done`.
FAIL	The builder fixes exactly what failed and the verifier checks again. After two failed rounds the task is `blocked`.
UNVERIFIED	Everything else passed, but a `[browser]` criterion couldn't be seen (no dev server or Playwright). The task goes to `review`, not `done`. Its log names what to check; when you've checked it, run `tasks.sh accept <id>` in your own terminal. Tasks that depend on it wait until then.
SKIPPED	Only for tiny changes: every criterion is `[review]` and the diff is at most 20 lines.

7. Running many tasks safely

Running tasks one after another in a single conversation lets context pile up, and quality drops by the 4th or 5th task. So `/laravel:task all` stops after 3. For longer batches use the runner, which starts a fresh session for every task:

```
bash .claude/hooks/run-tasks.sh --checkpoint              # all runnable tasks, in order
bash .claude/hooks/run-tasks.sh 004 005 --budget 3        # just these, max $3 each
```

Option	Effect
`--checkpoint`	One local git commit per finished task, on a new branch `nw/batch-<time>` so main is never touched. Each commit holds only that task's files: never `.env` or keys, never edits that were already there.
`--no-branch`	With `--checkpoint`, commit on the current branch instead
`--budget USD`	Spending limit per task
`--max N`	Stop after N tasks
`--dry-run`	Show what would run, and run nothing

The runner stops at the first task that doesn't finish (blocked, failed or over budget); run it again and it continues from there. A task that ends in `review` doesn't stop the batch: the runner tells you to check it and moves on to tasks that don't depend on it.

Undo one task (needs `--checkpoint`): `bash .claude/hooks/tasks.sh undo 004`. If later finished tasks depend on it, add `--cascade` to undo those first, newest first.

When the batch looks good, run `/laravel:sync` (section 8). Claude can't push or open pull requests on its own.

8. Git: one command to commit, pull, push

One command does the whole round. Only you can start it, so Claude never runs it on its own: **new**

```
/laravel:sync [commit message]
```

Step	What happens
1. Commit	Stages everything not ignored, refuses secret files, and commits with a message in your project's style (or yours, if you pass one).
2. Pull	Brings this branch's own remote commits and then the **stage branch** into the current branch (merge by default). Clear conflicts are resolved; unclear ones stop everything and are explained.
3. Check	If anything came in, the tests run again. A failure stops the push.
4. Push	Pushes the branch. It never pushes a branch that doesn't contain the stage branch's latest commit, and never force-pushes.

1 **Tell it your stage branch once**

```
./install.sh /path/to/app --stage-branch stage          # at install, or:
/laravel:setup                                             # asks, suggesting
stage/staging/develop
```

It's stored in `.claude/workflow.conf` (`STAGE_BRANCH=stage`, `SYNC_MODE=merge`). Commit that file so the team shares it. If it's missing, the first `/laravel:sync` asks.

A plain `git push` from Claude is always blocked and redirected, so no push can skip the stage pull. The git work is done by `git-sync.sh` (`status`, `sync`, `push`, `stage`, `candidates`), which you can also run in your own terminal. With the Laravel kit, a push happens only after **you** type `/laravel:sync` (that counts as your go-ahead for that turn only); asking in plain words commits but doesn't push.

9. All commands and helpers

Slash command	Model	Use it to…
`/laravel:setup`	current	Connect the workflow to the project. Run once.
`/laravel:write-task <what>`	Opus	Plan work into draft tasks. Never writes code.
`/laravel:task [id|all|list]`	Sonnet	Build tasks: your main command.
`/laravel:feature <id>`	Sonnet	Force the full phased build (migration → model → route → view → tests → review).
`/laravel:test [target]`	current	Write or fix tests outside a task.

Slash command	Model	Use it to…
`/laravel:review [target]`	Opus reviewer	Review changes for authorization, injection, caching, N+1 and Blade escaping bugs.
`/laravel:audit [path]`	current	Performance and accessibility audit, ranked.
`/laravel:ship`	current	Tests, lint, static analysis, asset build and deploy steps. Never commits.

Terminal helpers are below with the project-files path. With the plugin, use the **Kit helpers** path from the session report instead of `.claude/hooks`.

Terminal helper	Does
`bash .claude/hooks/tasks.sh list`	Status of every task
`… tasks.sh approve <id|all>`	Draft → todo, after you've read it (your terminal only)
`… tasks.sh accept <id>`	Review → done, after you checked the UNVERIFIED criteria (your terminal only)
`… tasks.sh diff <id>`	Everything a task changed
`… tasks.sh undo <id> [--cascade]`	Revert a task's checkpoint commit (and its dependents)
`… tasks.sh map`	A short, cached map of the app: models with relations, controllers, routes, tables, views
`… tasks.sh context <id>`	The task plus what its dependencies did and changed
`… tasks.sh stats`	Tasks done, time per task, batch cost, verifier first-pass rate, which checks fire most
`… tasks.sh dir / migrate`	Which folder holds the tasks / move an older project's `tasks/` to `.claude/tasks/`
`bash .claude/hooks/run-tasks.sh`	Batch runner, with a fresh session per task

10. The specialist agents

Agent	Job	Can edit?
`laravel-route-builder`	Routes, FormRequests, controllers, middleware, policies	Yes
`laravel-data-layer`	Migrations, models, queries, services, caching, jobs	Yes
`blade-ui-builder`	Views, components, Alpine, styling, the block-editor island	Yes
`laravel-test-writer`	Pest/PHPUnit tests	Yes
`laravel-verifier`	Checks every "done" criterion against the task and diff only	Throwaway browser tests only
`laravel-reviewer`	Security and correctness review (always Opus)	No
`laravel-perf-auditor`	Queries, indexes, caching, queues, asset size	No

Agent	Job	Can edit?
`blade-a11y-auditor`	Semantics, keyboard, labels, focus, contrast	No

Every agent starts with the stack report and project memory, so it doesn't explore your code from scratch. With the plugin, agents are named `laravel:laravel-verifier` and so on.

11. Automatic checks (hooks)

Check	When	What it does
`session-context`	Session start	Reports the kit version, the Kit helpers path and your stack, points to the Laravel 13 notes, and loads the project memory and task counts. Reloaded after compaction.
`guard-php`	After edits	Blocks `env()` outside config, mass assignment from the raw request, raw SQL built from variables, `dd()`, Laravel 8/9 file structure, SSRF and path traversal from request input, `upsert()` without its unique columns. Advises on N+1, Redis `KEYS`, `Cache::flush()`, unsafe upload names and image work in the request.
`guard-blade`	After edits	Blocks a missing `@csrf`, unescaped request data, and queries in views.
`guard-styles`	After edits	Checks Tailwind classes against the version you actually have installed.
`guard-editor`	After edits	Flags CKEditor/TinyMCE/Summernote/Quill in Post/Page code.
`guard-comments`	Before edits	Code written for a task carries no comments. Blocks an edit that adds a comment line, before the file is written. Comments already in the file, PHP attributes, tooling directives (`@phpstan-ignore`, `@ts-expect-error`) and type-only docblock tags (`@return HasMany<Post, $this>`) are allowed. 2.4
`format`	After edits	Runs your project's own Pint (or Prettier for JS/CSS).
`bash-guard`	After shell commands	Files written through the shell get the same checks.
`guard-secrets`	Before shell commands	Blocks commands that read `.env` files, keys or `auth.json`. `.env.example` is allowed. It slows accidents down but isn't a security boundary (section 16).
`guard-owner`	Before shell commands	Keeps your decisions yours: blocks `git push`, `gh pr create` / `merge`, `tasks.sh approve` / `accept`, shell edits of a task's status, and `verify.sh --baseline`. 2.3
`verify`	Before Claude finishes	`php -l` on changed PHP; optionally Pint and PHPStan (only new findings fail).
`guard-read`, `allow-helpers`	Plugin only	Stand in for the permission rules a plugin can't carry: block reading secret files, pre-approve the kit's own helpers, verify commands and Boost's read-only tools. Anything chained or unusual still asks you. 2.4

Every check hit is recorded in `.claude/.cache/metrics.jsonl`. `tasks.sh stats` summarises them.

12. Post and Page content: Gutenberg

Every Post, Page, News or Article module uses WordPress's own block editor (`@wordpress/block-editor`), mounted as a small React component in your Blade form through Vite. It never uses CKEditor or TinyMCE.

- **Storage:** `content` holds the block format (exactly like WordPress); `content_html` is a sanitized copy made when you save.
- **Display:** the public page outputs `{!! $post->content_html !!}`, the only unescaped output allowed.
- **Images:** they upload through a protected Laravel route to your disk or CDN.
- **Tests:** the planner adds them automatically: harmful HTML is stripped, authors can't edit each other's posts, and the editor saves a heading, a paragraph and an image.

Why not Laraberg? Its last release was in January 2023, and it relies on React 17, jQuery and Moment.

13. Redis, uploads, S3 and CDN

- **Redis:** phpredis is preferred (predis if your project already uses it). Cache data lives on its own connection. Every cached value needs a way to be cleared. Use tags only on Redis or Memcached. Never use `KEYS` or `Cache::flush()` on a shared Redis. On Laravel 13, cache arrays, not models.
- **Uploads:** validate with `File::image()->max('5mb')->dimensions(…)` (SVG is rejected by default). Store with random names, never the client's file name.
- **Public vs private:** public files use `storePublicly` and are served from the CDN. Private files are stored privately and served with `temporaryUrl()` after a permission check.
- **CDN:** set `AWS_URL=https://cdn.example.com` and `Storage::url()` returns CDN links. Store paths in the database, never full URLs. Never-changing file names with long cache headers mean you never have to purge the CDN.
- **Images:** resize, convert to WebP and create size variants in a queued job, not during the request.

14. Settings you can change

Set these in your shell before starting `claude` (or before the batch runner), or in the `env` section of your Claude Code settings.

Variable	Values	Effect
`LARAVEL_WF_CHECKPOINT`	`commit`	One local commit per finished task (the runner's `--checkpoint` sets this)
`LARAVEL_WF_DEV_URL`	e.g. `http://localhost:8000`	Lets the verifier check `[browser]` criteria (needs `@playwright/test`; the kit brings its own config)

Variable	Values	Effect
`LARAVEL_WF_VERIFY`	`syntax` (default), `syntax,lint,analyse`, `off`	What runs before Claude finishes
`LARAVEL_WF_REQUIRE_VERDICT`	`on`, `off`	The verdict gate on close (section 6)
`LARAVEL_WF_ALLOW_PUSH`	`1`	Let Claude push (still through `git-sync.sh`) and open/merge pull requests without you typing `/laravel:sync`
`LARAVEL_WF_STAGE_BRANCH`, `…_SYNC_MODE`	a branch; `merge`, `rebase`	Override `.claude/workflow.conf` (section 8)
`LARAVEL_WF_PHP_GUARD`, `…_BLADE_GUARD`	`warn`, `off`	Soften a guard
`LARAVEL_WF_EDITOR_GUARD`, `…_STYLE_GUARD`, `…_SECRETS_GUARD`	`off`	Turn a guard off
`LARAVEL_WF_COMMENT_GUARD`	`on`, `off`	The no-comments check (section 11)
`LARAVEL_WF_TASKS_DIR`	a folder	Keep tasks somewhere other than `.claude/tasks`
`LARAVEL_WF_METRICS`	`on`, `off`	Recording to `metrics.jsonl`

Update: plugin: `claude plugin update laravel@laravel-workflow`. Project files: run `./install.sh /path/to/app` again. **Remove:** `claude plugin uninstall laravel@laravel-workflow`, or `./install.sh /path/to/app --uninstall`, which removes the kit's files and hook entries and keeps your tasks, memory and own settings.

15. A typical day

1. `/laravel:write-task …`, then answer its questions.
2. Read the drafts in `.claude/tasks/`, then `tasks.sh approve all` in your own terminal.
3. Optional: start `php artisan serve` and set `LARAVEL_WF_DEV_URL` so browser checks really run.
4. `bash .claude/hooks/run-tasks.sh --checkpoint` and let it work, or `/laravel:task` for one at a time.
5. Check the result. For tasks in `review`, check the criteria named in their log, then `tasks.sh accept <id>`. Undo any task you don't like with `tasks.sh undo <id>`.
6. `/laravel:review`, then `/laravel:ship`. When everything is green, `/laravel:sync`: it pulls the stage branch in, re-runs the tests and pushes.
7. Now and then, run `tasks.sh stats` to see cost, time, the verifier's pass rate and which checks fire most.

16. Problems and fixes

Problem	Fix
Every turn fails on old PHPStan findings	Record them once: `! bash .claude/hooks/verify.sh --baseline`
"task is still a draft"	Read it, then `tasks.sh approve <id>` in your own terminal.
"no verdict recorded" when closing	Intended: every task needs the verifier's verdict (section 6). Let `/laravel:task` finish the verification step.
A task waits in `review`	A `[browser]` criterion couldn't be checked. Check it yourself, then `tasks.sh accept <id>`.
The verifier marks `[browser]` items UNVERIFIED	Start the app, set `LARAVEL_WF_DEV_URL`, and install Playwright.
Claude was blocked from pushing	Intended: type `/laravel:sync`, or run `git-sync.sh push` yourself.
`/laravel:sync` stopped on a conflict	The stage branch changed the same lines. Resolve them (or ask Claude to), then run `/laravel:sync` again. Nothing was pushed.
"installed twice" at session start	The project has both the plugin and project files. Keep one: `./install.sh /path/to/app --uninstall`, or disable the plugin.
Plugin: a helper command still asks permission	Only plain helper and verify commands are pre-approved. Anything chained, with flags, or matching one of your own ask rules asks on purpose.
`undo` says there's no checkpoint	The task was closed without `--checkpoint`. Use `tasks.sh diff` and revert by hand.
"checkpoint: SKIPPED — secret files changed"	Add the file to `.gitignore`, then commit the task's files yourself. Nothing was committed.
Claude asks permission to edit a file in `.claude/`	Normal: Claude Code protects that folder. New drafts don't ask, because they're written by `tasks.sh new`. Approve edits you asked for, or pick "allow for this session".
Your tasks are still in `tasks/` after an upgrade	They keep working. Move them with `tasks.sh migrate`.
A plan file in `.claude/tasks/` is ignored	That's intended: only files starting with `---` frontmatter are tasks.
A command was blocked for touching `.env`	Intended. Run it yourself, or use `.env.example`. The guard can't catch everything: for real protection keep production secrets out of the project folder, or use Claude Code's sandbox (`/sandbox`).
Boost tools not used	Install Boost (section 4) and approve the `.mcp.json` server.
An edit was blocked for adding a comment	Intended: Claude rewrites it without the comment. To allow comments, set `LARAVEL_WF_COMMENT_GUARD=off`.
A guard flags code you need	Set that guard to `warn` or `off` (section 14).

17. What's new in 2.4 and 2.3

Change	What it means for you
Git commands 2.4	`/laravel:sync` commits, pulls the stage branch in, checks and pushes in one go. Every push pulls the stage branch in first (section 8).
Plugin install 2.4	Install once, update with one command, nothing copied into projects (section 3). The commands stay `/laravel:*`; the session report prints where the helpers are.
Plugin safety 2.4	Guards always run, even if a project file tries to replace one. The kit's helpers are approved only by the plugin's own absolute path, and verify commands only with plain file arguments.
Pure code 2.4	Code written for a task has no comments. The rule is in every builder's instructions, and `guard-comments` blocks an edit that adds one, so Claude rewrites it without.
Verdict gate 2.3	A task can't close without the verifier's verdict. A FAIL must be fixed and checked again first.
Review status 2.3	Work with unchecked `[browser]` criteria goes to `review`; you `accept` it. Dependent tasks wait; the runner moves on to independent ones.
Owner guard 2.3	Claude can't push, open or merge pull requests, approve its own drafts, accept its own work, or change a task's status around the helper.
Fixes	The secrets guard's message no longer lists your folder's files when it blocks a pattern like `.e*`.

Earlier releases

Version	Main change
2.2	Tasks live in `.claude/tasks/`, and drafts are written by `tasks.sh new` with no permission prompt per draft. Only files starting with `---` count as tasks.
2.1	Context memory: repo map, task handoff, subagents start with the stack report, memory reloaded after compaction.
2.0	The verifier agent, draft → approve, a fresh session per task, per-task commits and undo, the static-analysis baseline, metrics, Laravel 13 rules, Boost, Gutenberg.

Every change is listed in `CHANGELOG.md` in the kit.

18. Measuring the workflow

```
./test.sh                          # the kit's self-test, against throwaway fixtures
bash evals/run.sh --reference      # free: proves every eval check can pass on a real Laravel app (must be 5/5)
bash evals/run.sh                  # 3 real agent runs (costs API money)
```

Results land in `evals/results/`. Compare them before and after changing any command, agent or skill. A change that lowers the pass rate or raises the cost is a regression.